

SOFTWARE REFERENCE MANUAL

SYNERGY ENGINEERING LABS

Aegis PQVM

Blockchain & Virtual Machine Post-Quantum Integration Module

MODULE

aegis-pqvm

TARGET

Blockchain / Smart Contract VMs

VM RUNTIMES

EVM / Substrate / CosmWasm / Solana / Move

STATUS

✓ Production

⚖️ LEGAL NOTICE & INTELLECTUAL PROPERTY

Copyright © 2026 Synergy Engineering Labs. All rights reserved. Aegis is patent pending.
This document is confidential and proprietary. Distribution is exclusively through licensed channels. Not available via public package managers.

NAVIGATION

Table of Contents

1	About Aegis	§1
2	Module Overview	§2
3	Technical Architecture	§3
4	Supported Algorithms	§4
5	VM Integration Patterns	§5
6	API Reference	§6
7	Testing & Validation	§7
8	Security & Compliance	§8
9	Build & Release	§9

SECTION 1

About Aegis



Quantum-Safe Cryptography Platform

Aegis is Synergy Engineering Labs' patent-pending post-quantum cryptography platform delivering NIST-standardized quantum-safe algorithms across every major computing environment without requiring architectural rebuilds.

`aegis-pqvm` is the blockchain and smart contract virtual machine integration module. It enables post-quantum cryptography in deterministic VM environments (EVM, Substrate, CosmWasm, Solana, Move) with fail-closed ABI parsing and validation.

SECTION 2

Module Overview — `aegis-pqvm`

PQC for Blockchain & Smart Contracts



aegis-pqvm provides post-quantum cryptography integration for deterministic virtual machine runtimes. Uses AEG1 byte ABI for compact, fail-closed operation verification at smart contract boundaries across EVM, Substrate, CosmWasm, Solana, and Move.

Blockchain systems require deterministic execution: all validators must produce identical results. Key generation and encapsulation use randomness and must execute off-chain. Only verification operations (signature verify, KEM decapsulate) are deterministic and safe for on-chain execution.



Multi-VM Support

EVM, Substrate, CosmWasm, Solana, Move.
Unified AEG1 ABI.



Fail-Closed ABI

Payload-size and argument-count limits.
Invalid input returns error, never fabricated output.



Deterministic Execution

All validators produce identical results. No randomness in hot paths. Off-chain keygen/encap.



Verification Only

ML-DSA/FN-DSA signature verify. ML-KEM decapsulate. Deterministic on-chain ops.



Lifecycle Tracking



Production Quality

Key version management. Beacon integration. Runtime self-test helpers.

Full validation harness. Reproducible builds. pqcore source-root pinning.

SECTION 3

Technical Architecture

AEG1 ABI Specification

```
// AEG1 byte ABI for PQC operations  
// Compact binary ABI for passing operations through VM call boundaries
```

Operation Layout:

```
[1 byte]  opcode (MLKEM_DECAP=0x01, MLDSA_VERIFY=0x02, ...)  
[4 bytes] payload size (big-endian)  
[N bytes] operation payload (ciphertext, signature, public key, etc.)  
[1 byte]  argument count
```

Validation Rules:

- Max payload size: 16,384 bytes (prevents DoS)
- Max argument count: 255
- Invalid input: return error (fail-closed)
- No fabricated output on parse failure

Integration Shims

```
aegis-pqvm/  
├─ evm_precompile/           # EVM integration  
├─ substrate_pallet/         # Substrate integration  
├─ cosmwasm_module/          # CosmWasm integration  
├─ solana_program/           # Solana integration  
├─ move_module/              # Move integration  
├─ core/                     # AEG1 ABI implementation  
├─ docs/specification/        # TECHNICAL_SPECIFICATION.md  
├─ docs/developer/           # DEVELOPER_GUIDE.md  
├─ PQVM_USAGE_MANUAL.md      # Usage guide  
└─ run_quality_gates.sh      # PQVM-specific validation
```

Why Off-Chain Keygen, On-Chain Verify

Smart contracts must be deterministic. All validators independently execute the same code and must reach identical state transitions. Key generation inherently requires randomness—different random inputs produce different keys. Encapsulation requires random ephemeral values. Only verification is deterministic: given identical inputs (message, signature, public key), all validators produce identical output (valid/invalid).

SECTION 4

Supported Algorithms

Operation	Algorithm	NIST Std	On-Chain	Off-Chain	Status
Signature Verify	ML-DSA	FIPS 204	✓	—	✓

Operation	Algorithm	NIST Std	On-Chain	Off-Chain	Status
Signature Verify	FN-DSA	FIPS 206	✓	—	✓
KEM Decapsulate	ML-KEM	FIPS 203	✓	—	✓
Signature Generate	ML-DSA / FN-DSA	FIPS 204/206	—	✓	✓
KEM Encapsulate	ML-KEM	FIPS 203	—	✓	✓
Key Generation	All families	FIPS 203-207	—	✓	✓

SECTION 5

VM Integration Patterns

EVM Precompile Integration

Register PQC operations as EVM precompiles. Contracts call via standard `CALL` with AEG1-encoded input.

```
// Solidity: Verify ML-DSA signature via precompile
function verifyPQSignature(
    bytes calldata message,
    bytes calldata signature,
    bytes calldata publicKey
```

```

) external returns (bool) {
    bytes memory payload = abi.encodePacked(
        uint8(0x02), // MLDSA_VERIFY opcode
        uint32(message.length + signature.length + publicKey.length),
        message, signature, publicKey
    );

    (bool success, bytes memory result) = address(0xpgc).staticcall(payload);
    require(success, "PQC verification failed");
    return true;
}

```

Substrate Pallet Integration

Implement as runtime pallet. AEG1 ABI validation at pallet interface.

```

// Rust: Substrate PQC pallet
pub fn verify_pq_signature(
    origin: OriginFor<T>,
    message: Vec<u8>,
    signature: Vec<u8>,
    public_key: Vec<u8>,
) -> DispatchResult {
    let _who = ensure_signed(origin)?;

    // AEG1 ABI validation
    let payload_size = message.len() + signature.len() + public_key.len();
    ensure!(payload_size <= 16384, Error::PayloadTooLarge);

    // Call ML-DSA verify via pqcore
    let valid = aegis_pqvm::mldsa_verify_deterministic(
        &message, &signature, &public_key
    )?;

    ensure!(valid, Error::InvalidSignature);
    Ok(())
}

```

CosmWasm Module Integration

```

// CosmWasm: PQC verification contract
pub fn execute_verify(
    deps: DepsMut,
    _env: Env,
    _info: MessageInfo,
    msg: VerifyMsg,
) -> Result<Response, ContractError> {
    // AEG1 ABI encoding
    let payload = aegis_pqvm::aeg1_encode(
        AEG10p::MlDsaVerify,
        [&msg.message, &msg.signature, &msg.public_key],
    )?;

    let result = aegis_pqvm::verify_deterministic(&payload)?;

    Ok(Response::default()
        .add_attribute("pq_valid", result.to_string()))
}

```

Solana BPF Program

```

// Solana: PQC instruction handler
pub fn process_pq_verify(
    _program_id: &Pubkey,
    accounts: &[AccountInfo],
    instruction_data: &[u8],
) -> ProgramResult {
    // Parse AEG1 ABI
    let (opcode, payload) = aegis_pqvm::aeg1_parse(instruction_data)?;

    // Execute deterministic verification
    aegis_pqvm::verify_deterministic(opcode, &payload)?;

    Ok(())
}

```

SECTION 6

API Reference

AEG1 ABI Functions

```
// Rust FFI
aeg1_encode(opcode: u8, payloads: &[&[u8]]) -> Result<Vec<u8>>
aeg1_parse(data: &[u8]) -> Result<(u8, Vec<u8>>)

// Verification (deterministic, on-chain safe)
mldsa_verify_deterministic(msg: &[u8], sig: &[u8], pk: &[u8]) -> Result<bool>
fndsa_verify_deterministic(msg: &[u8], sig: &[u8], pk: &[u8]) -> Result<bool>
mlkem_decapsulate_deterministic(sk: &[u8], ct: &[u8]) -> Result<Vec<u8>>
```

Lifecycle Management

```
lifecycle_create_manager() -> LifecycleManager
lifecycle_rotate_key(variant: &str) -> Result<KeyPair>
lifecycle_get_version() -> u32
```

Self-Test Helpers

```
runtime_self_test() -> Result<()>
beacon_generate() -> Result<Vec<u8>>
beacon_verify_latest(data: &[u8]) -> Result<bool>
```

SECTION 7

Testing & Validation

Quality Gates

ci_security_baseline.sh

run_validation_harness.sh

replay_validation_harness.sh

run_fuzz_smoke.sh

build_release_bundle.sh

Validation Harness

```
./run_validation_harness.sh    # Full AEG1 ABI validation
./replay_validation_harness.sh # Replay saved test cases
./run_fuzz_smoke.sh           # Fuzz ABI parser
```

Determinism Testing

All on-chain operations are validated for strict determinism. Same input on all validators produces identical output. No randomness in hot paths. Off-chain key generation is user's responsibility.

SECTION 8

Security & Compliance

Deterministic Execution Guarantees

- No randomness in verification code paths
- No time-dependent logic
- No network I/O
- No filesystem access
- Identical output across all validators

AEG1 ABI Validation

- Payload-size limit: 16,384 bytes (prevents DoS)
- Argument-count limit: 255
- Fail-closed parsing: invalid input returns error, never fabricated output
- Type-safe bounds checking in all VM contexts

Source-Root Pinning

Reproducible builds via `AEGIS_PQ_SOURCE_ROOT` environment variable. Builds can be verified to use exact pqcore source code. All validators can validate they're running identical PQC implementation.



Off-Chain Key Generation Required

Key generation **MUST** occur off-chain. Smart contracts should only verify signatures or decapsulate KEMs. Attempting keygen in deterministic VM context will fail.

SECTION 9

Build & Release

Build Process

1

Prepare pqcore

Set `AEGIS_PQ_SOURCE_ROOT` to exact pqcore directory. Reproducible builds require exact source paths.

2

Run validation harness

`./run_quality_gates.sh` validates all AEG1 ABI operations, determinism constraints, and integration shims.

3

Fuzz ABI parser

`./run_fuzz_smoke.sh` generates malformed AEG1 payloads to verify fail-closed behavior.

4

Build release bundle

`./build_release_bundle.sh` creates VM-specific integration packages.

VM-Specific Deployment

- **EVM** — Deploy as precompile contract. Address: 0xppc (to be assigned)
- **Substrate** — Include pallet in runtime, enable in genesis config
- **CosmWasm** — Deploy contract, use for message verification
- **Solana** — Deploy program, invoke via instruction
- **Move** — Include module in framework, call via entry function



Production Ready

`aegis-pqvm` has completed all production quality gates. Approved for blockchain deployment. See PQVM_USAGE_MANUAL.md for deployment best practices per VM.

Aegis PQVM Software Manual

© 2026 Synergy Engineering Labs. All rights reserved.

Patent Pending. Confidential & Proprietary.